

Midterm 2: Concurrency

Time: 90 minutes Total Questions 63 / total points 80

Questions in parts 1 and 2 are 1 point each; questions in parts 3 and 4 are worth 2 points each.

Instructions: This exam is a closed book exam. You are allowed one 8.5"x11" sheet of paper and you must turn it in with your exam. You will hand the question set and the scantron sheet back to the instructor after the exam is over.

Please fill in your name and student ID on the answer sheet. There is no special code. Please use a pencil to fill in the bubbles

When you begin the exam, you will need to agree with the following statements:

- I understand that this exam must be worked on independently.
- I will not communicate or collaborate with any others during the exam in any way; for example:
 - I will not chat, text, or post any questions or information.
 - I will not ask for, receive, or provide help to others.
 - I will notify the instructor if I am aware of others violating this policy.
- I will not discuss the questions with students of the other section.

Unless stated (or implied) otherwise, you should make the following assumptions:

- The OS manages a multiprocessor (multi-core) that can run multiple threads simultaneously
- Leading zeros can be removed from numbers (e.g., 0x06 == 0x6)
- Hex numbers are represented with a preceding "0x"
- All the code mentioned in the paper is assumed to compile properly. You can ignore any syntax error if there is any.

Section - 1: True/False questions. There are 25 questions, and each question is worth 1 point. On the answer sheet, True = A, False = B.

1. Given that threads A and B are part of the same process and run on the same core, threads A and B can use the same TLB entry to speed up their address translations.
2. Threads that are part of the same process share the same heap.
3. With kernel-level threads, if one thread of a process blocks, all the threads of that process will also be blocked.
4. Locks prevent the OS scheduler from performing a context switch during a critical section.
5. All spinlocks provide bounded waiting.
6. A context switch can occur in the middle of the atomic hardware instruction XCHG.
7. The “ticket lock” can be implemented using spin-waiting or blocking.
8. A lock implementation should block instead of spin if it will always be used only on a uniprocessor.
9. When a thread returns from a call to `cond_wait()` it can safely assume that it holds the corresponding mutex.
10. A thread can hold only one lock at a time.
11. A semaphore can be used to provide mutual exclusion.
12. The performance of broadcasting to a condition variable decreases as the number of waiting threads increases.
13. As the amount of code a mutex protects increases, the amount of concurrency in the application increases.
14. A wait-free algorithm relies on condition variables instead of locks.
15. Deadlock can be avoided if threads acquire all potentially needed locks atomically.
16. Two-phase waiting assumes that an oracle can predict the amount of time that the lock will be held by another process.
17. Condition variables can provide both ordering and mutual exclusion.
18. The counter in semaphores can be accessed by the user program.
19. Semaphores can be used for both mutual exclusion and ordering.
20. Using atomic instructions instead of using locks can help eliminate deadlocks.
21. To avoid deadlock, if a thread cannot get the resources it wants, it may release all the resources it holds.
22. When the critical section is very large/long, it is a good idea to block the waiting thread and put that thread on a waiting queue instead of spinning.

In the next four True/False questions, two processes, A and B, which share semaphores R and S, are executing the following code:

```
sem_init(&R, 0);
sem_init(&S, 1);
void process_A() {
    sem_wait(&R);
    sem_wait(&S);
    printf("A.1");
    sem_post(&S);
    printf("A.2");
    sem_post(&R);
}

void process_B() {
    printf("B.1");
    sem_post(&R);
    printf("B.2");
    sem_wait(&S);
    printf("B.3");
    sem_post(&S);
}
```

- 23. "A.1" will always be printed after "B.1"
- 24. "A.1" and "B.2" can be printed in any order
- 25. "A.1" and "B.3" can be printed in any order
- 26. "A.2" will always be printed before "B.3"

Part 2: multiple choice questions. There are 20 questions worth 1 point each

27. Which statement is **not true** about implementing locks by disabling interrupts?
- a. It only works on uniprocessors
 - b. Processes can keep control of CPU for an arbitrary amount of time
 - c. A thread holding the lock can be descheduled if it exhausts its timeslice.
 - d. It does not provide bounded waiting.
28. Suppose you want to use a semaphore as a binary semaphore to implement a lock. What value should you pass to `sem_init()`.
- a. -1
 - b. 0
 - c. 1
29. Suppose you want to use a semaphore to implement `join()` for one waiting thread. What value should you pass to `sem_init()`.
- a. -1
 - b. 0
 - c. 1
30. You want to implement a bounded buffer with two producers and two consumers using two semaphores, `semFull` and `semEmpty`. `semFull` is used to wait for data to be produced, and `semEmpty` is used to wait for buffers to be consumed/emptied. Suppose you want a buffer of size 3. How should you initialize the two semaphores?
- a. `semFull = 0; semEmpty = 0;`
 - b. `semFull = 3; semEmpty = 0;`
 - c. `semFull = 0; semEmpty = 3;`
 - d. `semFull = 3; semEmpty = 3;`
31. The following implementation of `thread_join()` and `thread_exit()` is a little bit different from what you saw in lecture. Specifically, lines c2 and c3 have been swapped. Do they still work correctly?

```
void thread_join() {
    Mutex_lock(&m); // p1
    while (done == 0) // p2
        Cond_wait(&c, &m); // p3
    Mutex_unlock(&m); // p4
}

void thread_exit() {
    Mutex_lock(&m); // c1
    Cond_signal(&c); // c2
    done = 1; // c3
    Mutex_unlock(&m); // c4
}
```

- a. Only if the parent calls `thread_join()` and `cond_wait()` before the child calls `thread_exit()`
- b. Works correctly in all cases
- c. Only if the child completes `thread_exit()` before the parent calls `thread_join()`
- d. Only if there is not a context switch between when the child calls `cond_signal()` and sets `done = 1`

32. You and your project partner are working on a project and want to talk. Both of you decide to call at the exact same time and find the other person is busy. So, both of you hang up and call again simultaneously. Both of you keep on doing the same again and again. What is this condition called?

- a. Deadlock
- b. Livelock
- c. Deadlock & Livelock
- d. Neither a deadlock nor a livelock.

33. With this code, what is printed out?

```
void * worker(void * arg) {
    int * i = (int *) arg;
    *i = *i + 1;
    pthread_exit(NULL);
}

int main(int argc, char * argv) {
    int x = 1;
    pthread_t p1;
    pthread_create(&p1, NULL, worker, (void *) &x);
    x = x + 1;
    printf("%d", x);
}
```

- a. 1
- b. 2
- c. 3
- d. 2 or 3
- e. 1 or 2

34. Suppose you use a blocking lock in user-mode code on a uniprocessor on a data structure that is highly contended and 10 threads are trying to access the data structure. On a system where a context switch takes 10 microseconds and a time slice is 1 millisecond, what answer best represents the amount of time the system spends on the waiting threads?

- a. $O(1 \text{ microsecond})$
- b. $O(10 \text{ microseconds})$
- c. $O(10 \text{ milliseconds})$
- d. $O(10 \text{ threads} * 10 \text{ microseconds})$
- e. $O(10 \text{ threads} * 1 \text{ milliseconds})$

35. Consider the following code snippet.

```
void child(void *arg) {

    // do some stuff, then signal completion
    sem_post(&s)
}

int main(int argc, char *argv[]) {
    thread_t p1, p2, p3, p4;
    sem_init(&s, ???);
}
```

```

        thread_create(&p1, child);
        thread_create(&p2, child);
        thread_create(&p3, child);
        thread_create(&p4, child);
        sem_wait(&s);
    }

```

What should the semaphore be initialized to such that the parent thread waits for all four child threads?

- a. 1
- b. 4
- c. 0
- d. none of the above

36. You want to use condition variables to implement a traffic light controller for standard traffic light with three colors (red, green, yellow). Waiting cars can only go when the light is green. Which of these is a possible correct use of condition variables?

- a. if (light == green) cond_wait(cv, lock);
- b. while (light == red) cond_wait(cv, lock);
- c. while (light != green) cond_wait(cv, lock);
- d. if (light == red) cond_wait(cv, lock);

37. Consider the following code snippet

```

int bytesLeft = MAX_HEAP_SIZE;
cond_t c;
mutex_t m;

void* allocate(int size) {
    Pthread_mutex_lock(&m);
    while (bytesLeft < size)
        Pthread_cond_wait(&c, &m);

    void *ptr = ...; // get mem from heap
    bytesLeft -= size;
    Pthread_mutex_unlock(&m);
    return ptr;
}

void free(void *ptr, int size) {
    Pthread_mutex_lock(&m);
    bytesLeft += size;
    Pthread_cond_signal(&c);
    Pthread_mutex_unlock(&m);
}

```

Assume there is no space left on the heap at the beginning (bytesLeft = 0). Three threads are run in the following order. T1: allocate(100) and then T2: allocate(20) and then T3: free(30) Which of these is a possible final outcome?

- a. Both T1 and T2 are blocked
- b. T2 alone might be blocked after this sequence
- c. No threads will be blocked after this sequence
- d. T3 alone might be blocked after this sequence

38. Assume there is no space left on the heap at the beginning (bytesLeft = 0). Three threads are run in the following order. What is the final outcome? T1: allocate(50) and then T2: allocate(30) and then T3: free(50)

- a. Either T1 or T2 might be blocked at the end of this sequence
- b. No threads will be blocked after this sequence
- c. Either T1 or T3 might be blocked at the end of this sequence
- d. Either T2 or T3 might be blocked at the end of this sequence

39. No question. Just answer A

Your project partner has written the following correct thread-safe implementation of add() using traditional locks for mutual exclusion:

```
void add (int *val, int amt) {  
    mutex_lock(&m);  
    *val += amt;  
    mutex_unlock(&m);  
}
```

You decide you would like to replace the lock with calls to the atomic hardware instruction CmpAndSwap(int *addr, int expect, int new), which returns 0 on failure and 1 on success. The exact behavior of atomic CompAndSwap() is defined as in class. You know that add() modified to use CmpAndSwap() looks something like the following [5 questions]:

```
void add (int *val, int amt) {  
    do {  
        int old = *val;  
    } while (CmpAndSwap(<Q1>, <Q2>, <Q3>) == <Q4>);  
}
```

What are the correct replacements for the symbols < Q1, Q2, Q3, Q4> in the code above?

40. <Q1> should be replaced with

- a. val
- b. *val
- c. old
- d. old + amt
- e. None of the above

41. <Q2> should be replaced with

- a. val
- b. *val
- c. old
- d. old + amt
- e. None of the above

42. <Q3> should be replaced with

- a. val
- b. *val
- c. old
- d. old + amt
- e. None of the above

43. <Q4> should be replaced with

- a. 0
- b. 1
- c. val
- d. *val
- e. None of the above

44. Assume that the new implementation of `add` (that uses `CmpAndSwap`) is correct. If 5 threads call `add`, for a given thread, in the worst-case scenario, on which attempt can `CmpAndSwap` successfully add `amt` to `val`?"

- a. Second attempt
- b. Third attempt
- c. Fourth attempt
- d. Fifth attempt
- e. There is no bound on the number of attempt

You are given the following code snippet which avoids deadlocks by using a global or metalock while adding values from src to dst. However some parts of the code are missing.

[2 questions]

```
void vector_add(vector_t *v_dst, vector_t *v_src) {
    // put GLOBAL lock around all lock acquisition...
    Pthread_mutex_lock(&global);
    Pthread_mutex_lock(&v_dst->lock);
    Pthread_mutex_lock(&v_src->lock);
    ??????????????????????????????
    int i;
    for (i = 0; i < VECTOR_SIZE; i++) {
        v_dst->values[i] = v_dst->values[i] +
                        v_src->values[i];
    }
    XXXXXXXXXXXXXXXXXXXXXXXX
    YYYYYYYYYYYYYYYYYYYYYYYY
}
```

45. What line of code should be used to fill in ??????????????????????????????
- Pthread_mutex_unlock(&global);
 - Either Pthread_mutex_unlock(&v_src->lock); or Pthread_mutex_unlock(&v_dst->lock);
 - Pthread_mutex_unlock(&v_dst->lock);
 - Pthread_mutex_unlock(&v_src->lock);
46. What line of code should be used to fill in YYYYYYYYYYYYYYYYYYYYYYYY
- Either Pthread_mutex_unlock(&v_src->lock); or Pthread_mutex_unlock(&v_dst->lock);
 - Pthread_mutex_unlock(&v_dst->lock);
 - Pthread_mutex_unlock(&v_src->lock);
 - Pthread_mutex_unlock(&global);

Part 3: Impact of scheduling without locks. There are 8 questions worth 2 points each.

For the next questions, assume that two threads are running the following code on a uniprocessor.

```
# assumes %bx has loop count in it
.top # start of loop
1000:    mov 2000, %ax # get the value at the address 2000
1001:    add $1, %ax   # increment it
1002:    mov %ax, 2000 # store it back to address 2000
# see if we're still looping
1003:    sub $1, %bx # Subtract 1 from bx
1004:    test $0, %bx # Compare bx against zero
1005:    jgt .top # jump to start of loop of bx > zero
1006:    halt
```

This code is incrementing a variable (e.g., a shared balance) in a loop. Assume that the %bx register begins with the value 3, so that each thread performs the loop 3 times. Assume the code is loaded at address 1000 and that the memory address 2000 originally contains the value 0. Assume that the scheduler runs the two threads producing the following order of instructions (the first column shows the address of the executed instruction). The code continues on the next page.

For each of the lines designated below, determine the contents of the memory address 2000 **AFTER** that assembly instruction executes. The answer to each of the following questions are one of these five possible values.

- a. 1
- b. 2
- c. 3
- d. 4
- e. 5

	Thread 0	Thread 1
	1000 mov 2000, %ax	
	1001 add \$1, %ax	
	----- Interrupt -----	----- Interrupt -----
		1000 mov 2000, %ax
		1001 add \$1, %ax
	----- Interrupt -----	----- Interrupt -----
47. Content of addr 2000?	1002 mov %ax, 2000	
	----- Interrupt -----	----- Interrupt -----
		1002 mov %ax, 2000
		1003 sub \$1, %bx
		1004 test \$0, %bx
		1005 jgt .top
48. Content of addr 2000?	1000 mov 2000, %ax	1000 mov 2000, %ax
	----- Interrupt -----	----- Interrupt -----
	1003 sub \$1, %bx	
	1004 test \$0, %bx	
	1005 jgt .top	
	1000 mov 2000, %ax	
	----- Interrupt -----	----- Interrupt -----
		1001 add \$1, %ax
49. Content of addr 2000?		1002 mov %ax, 2000
	----- Interrupt -----	----- Interrupt -----
	1001 add \$1, %ax	
50. Content of addr 2000?	1002 mov %ax, 2000	
	1003 sub \$1, %bx	
	1004 test \$0, %bx	
	1005 jgt .top	
	1000 mov 2000, %ax	
51. Content of addr 2000?	1001 add \$1, %ax	
	----- Interrupt -----	----- Interrupt -----
		1003 sub \$1, %bx
		1004 test \$0, %bx
		1005 jgt .top
		1000 mov 2000, %ax
52. Content of addr 2000?		1001 add \$1, %ax
	----- Interrupt -----	----- Interrupt -----
53. Content of addr 2000?	1002 mov %ax, 2000	
	----- Interrupt -----	----- Interrupt -----
54. Content of addr 2000?		1002 mov %ax, 2000
		1003 sub \$1, %bx
		1004 test \$0, %bx
		1005 jgt .top
		1006 halt
	----- Interrupt -----	----- Interrupt -----
	1003 sub \$1, %bx	
	1004 test \$0, %bx	
	1005 jgt .top	
	1006 halt	

Part 4: Reader/Writer Locks with Semaphores. There are 9 questions worth 2 point each

In this question, Assume the code compiles correctly and ignore any incorrect capitalization. The variable names explain the counters used to keep track of readers and writers. The semaphore names explain the use of semaphores.

```
Acquire_readlock() {
    Sem_wait(&mutex); // AR1 (line 1 of Acquire_readlock)
    if (ActiveWriters + // AR2
        WaitingWriters==0) { // AR3
        sem_post(OKToRead); // AR4
        ActiveReaders++; // AR5
    } else WaitingReaders++; // AR6
    Sem_post(&mutex); // AR7
    Sem_wait(OKToRead); // AR8
}

Release_readlock() {
    Sem_wait(&mutex); // RR1 (line 1 of Release_readlock)
    ActiveReaders--; // RR2
    if (ActiveReaders==0 && // RR3
        WaitingWriters > 0) { // RR4
        ActiveWriters++; // RR5
        WaitingWriters--; // RR6
        Sem_post(OKToWrite); // RR7
    }
    Sem_post(&mutex); // RR8
}

Acquire_writelock() {
    Sem_wait(&mutex); // AW1 (line 1 of Acquire_writelock)
    if (ActiveWriters + ActiveReaders + WaitingWriters==0) { // AW2
        ActiveWriters++; // AW3
        sem_post(OKToWrite); // AW4
    } else WaitingWriters++; // AW5
    Sem_post(&mutex); // AW6
    Sem_wait(OKToWrite); // AW7
}
```

```

Release_writelock() {
    Sem_wait(&mutex); // RW1
    ActiveWriters--; // RW2
    if (WaitingWriters > 0) { // RW3
        ActiveWriters++; // RW4
        WaitingWriters--; // RW5
        Sem_post(OKToWrite); // RW6
    } else while(WaitingReaders>0) { // RW7
        ActiveReaders++; // RW8
        WaitingReaders--; // RW9
        sem_post(OKToRead); // RW10
    }
    Sem_post(&mutex); // RW11
}

```

55. Does this implementation of reader/writer locks give priority to readers or to writers? That is, if there is both a reader and a writer waiting to acquire this lock, which one will be given the lock next?

- a. The reader
- b. The writer
- c. The thread that called `acquire_readlock()` or `acquire_writelock()` first
- d. Strictly alternates between readers and writers
- e. Cannot determine because cannot make assumptions about behavior of scheduler

56. How should the semaphore `OKToRead` be initialized?

- a. 0
- b. 1
- c. To the number of reader threads
- d. To the number of writer threads
- e. None of the above

57. How should the semaphore `OKToWrite` be initialized?

- a. 0
- b. 1
- c. To the number of reader threads
- d. To the number of writer threads
- e. None of the above

Assume the following calls are made by threads in the system and all three threads execute as far along as they can until they execute a statement that causes them to block (or wait) and they acquire the mutex in the order the threads are executed. Assume that after a thread returns from one of these four functions, the thread executes other user code that does not involve any synchronization or blocking (i.e., some code beyond AR8, RR8, AW7, and RW11).

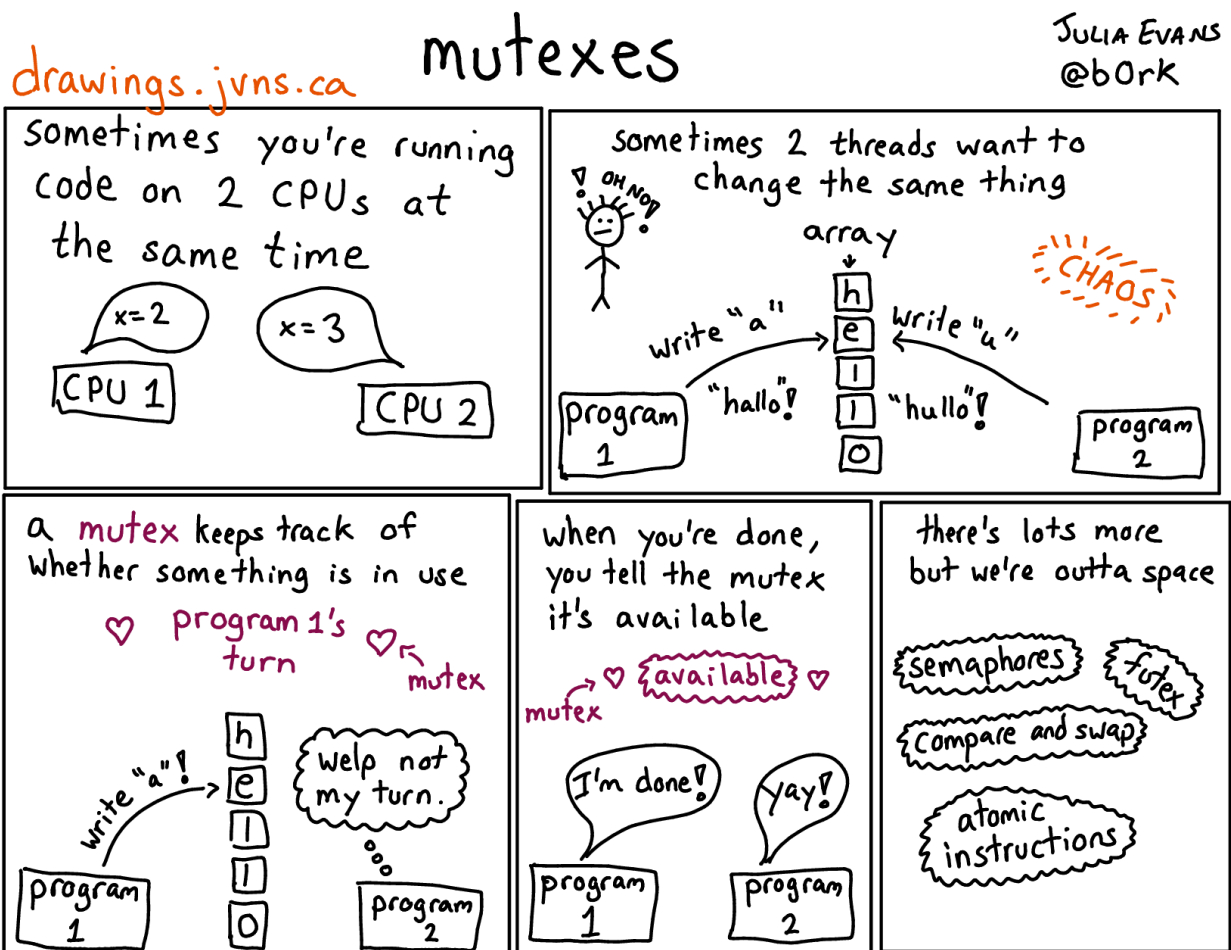
```
Reader Thread R0: Acquire_readlock();
Writer Thread W0: Acquire_writelock();
Reader Thread R1: Acquire_readlock();
```

58. Where will reader thread R0 be in the code?
- a. AR1
 - b. AR4
 - c. AR8
 - d. Beyond AR8
 - e. None of the above (including non-deterministic locations)
59. Where will thread W0 be in the code?
- a. AW1
 - b. AW4
 - c. AW7
 - d. Beyond AW7
 - e. None of the above (including non-deterministic locations)
60. Where will thread R1 be in the code?
- a. AR1
 - b. AR4
 - c. AR8
 - d. Beyond AR8
 - e. None of the above (including non-deterministic locations)

Continuing the same execution stream as above, now assume Thread R0 calls `release_readlock()` and the three threads again execute as far along as they can until they run into a statement that causes them to block (or wait).

61. Where will thread R0 be in the code?
- a. RR1
 - b. RR7
 - c. RR8
 - d. Beyond RR8
 - e. None of the above (including non-deterministic locations)

62. Where will thread W0 be in the code?
- AW1
 - AW4
 - AW7
 - Beyond AW7
 - None of the above (including non-deterministic locations)
63. Where will thread R1 be in the code?
- AR1
 - AR4
 - AR8
 - Beyond AR8
 - None of the above (including non-deterministic locations)



<https://wizardzines.com/comics/mutexes/>